pico-WAM: Design and Evaluation of a Compact World Action Model

Keivalya Pandya
Northeastern University

Abstract

World action models combine action generation with prediction of future observations or states. We present pico-WAM, a world action model of about a million parameters in which a shared latent supports a policy, an action-conditioned transition model, and state and image readouts. We study it in two environments whose simulators can replay any alternative action: point-mass navigation, and Push-T trained on its 206 human demonstrations. In both, state predictions along recorded actions are far more accurate than persistence, but predictions under other actions depend on the training data. Expert-only point-mass data gives about 15× higher counterfactual error than data with random-action bursts. On Push-T, a model trained on the human demonstrations predicts the block's response to new pushes worse than assuming that it stays put; branching the demonstrations in the simulator brings it to 0.45 times that reference with otherwise identical training. Planning with the learned dynamics near the policy's own plan outperforms the reactive policies, whereas a stronger search, or one warm-started from the previous plan, raises the improvement the model expects without improving the simulated outcome. The action head turns out to be the bottleneck: widening it from one hidden layer of 128 units to three of 512 raises planning from 77 to 121 successes in 300 unseen episodes and nearly closes the gap between the WAM's own policy and the behavioral-cloning baseline mini-VLA, which succeeds in 116. With images as the only input, the WAM succeeds in 46 episodes by planning and mini-VLA in 0. Matched pilots in both environments attribute most of the auxiliary gain to current-state supervision rather than future prediction, and image decoding accounts for most of the training cost but is not needed for planning.

1 Introduction

A vision-language-action (VLA) policy maps observations and instructions to actions. An action-conditioned world model predicts how the environment changes under a specified action. World action models (WAMs) combine these functions through joint prediction, inverse dynamics, or planning [13, 16]. We study joint training: a policy and forward model share a representation (Figure 1). The policy can act on its own, and on Push-T we also plan with the forward model.

Several recent WAMs use billion-parameter video backbones [8, 9, 11], making controlled comparisons expensive. Tutorials describe their design choices [18], and minimal video-prediction implementations simplify some components [6]. A small model containing both a policy and a predictor allows their interaction to be tested directly.

We present pico-WAM, a model of about a million parameters with image, instruction, and state encoders, an action head, latent dynamics, and state and image decoders:

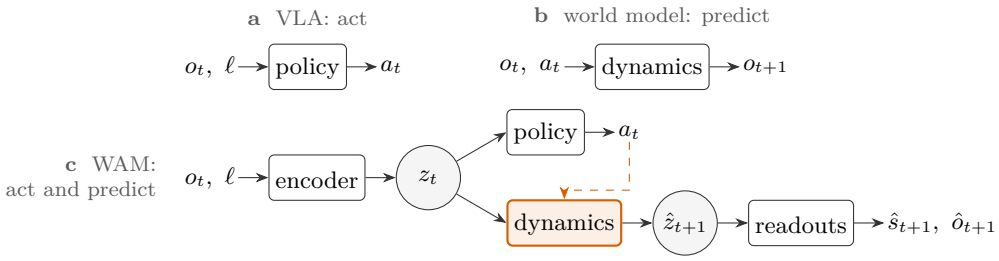


Figure 1: Policy prediction, forward prediction, and joint training. (a) A policy maps observation o_t and instruction ℓ to action a_t. (b) An action-conditioned forward model predicts a future observation. (c) In pico-WAM, a policy and transition model share latent state z_t. The transition model accepts a specified action, and readouts convert its predicted latent into a physical state and image. Repeated transitions generate an open-loop rollout. The reactive policy does not use this rollout to select actions; the Push-T planner does.

0.75M in the point-mass study, and 1.33M on Push-T, where a larger action head accounts for the difference. We study it in two environments whose simulators can replay any alternative action exactly. A point-mass navigation task has affine dynamics and a fully known state, which makes each effect easy to isolate. Push-T [4] requires pushing a T-shaped block onto a target through frictionless contact, and we train on its 206 human demonstrations. In both, we ask whether prediction along recorded trajectories generalizes to other actions, whether auxiliary prediction improves the policy, whether the world model is accurate enough to plan with, and which components account for the cost.

The paper compares eight open-source systems, evaluates data coverage and training objectives in both environments, tests planning with the learned dynamics on Push-T, and profiles parameters and runtime. The architecture uses established components; the contribution is an implementation and an empirical study of their behavior at a scale where every prediction can be checked against a simulator.

2 Prediction and control in existing systems

We compare systems along four dimensions: the prediction target, the relation between actions and predictions, the model architecture, and the use of predicted futures (Figure 2). These dimensions describe design choices rather than mutually exclusive model classes. For example, joint training can include an action-conditioned forward model, and a video model can serve both as a predictor and as a source of synthetic data. Table 1 includes such related systems alongside joint policy–prediction models.

These deployment roles require different evaluations. Auxiliary prediction should be judged by its effect on the deployed policy; planning additionally requires accurate comparisons between candidate actions. Synthetic-data methods must be evaluated through policies trained on their generated data. pico-WAM studies the first setting and tests its forward model separately.

GR-1's future-frame objective [17], Cosmos Policy's action selection [8], and V-JEPA 2-AC's representation-space planning [1] motivate our tests of prediction horizon, action coverage, and decoding cost. Planning with learned dynamics has also been studied on Push-T itself. DINO-WM plans with a cross-entropy method in the patch features

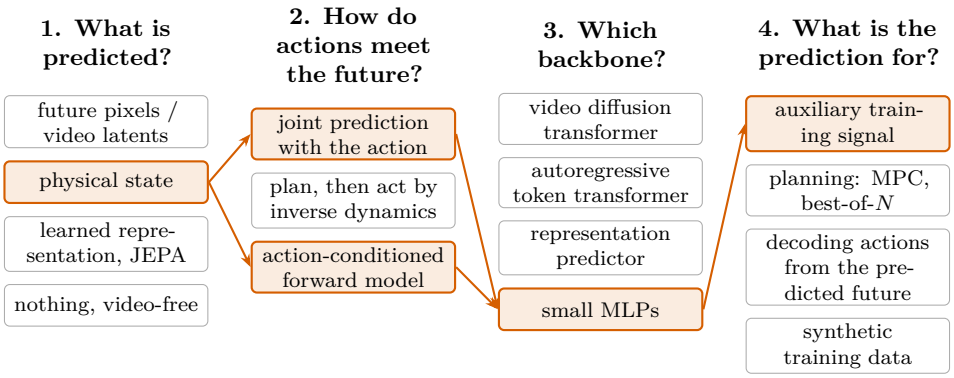


Figure 2: Design choices in predictive control systems. The columns distinguish prediction target, action modeling, architecture, and use of predictions. Orange marks pico-WAM's main configuration: state prediction, joint policy–dynamics training, an action-conditioned MLP transition, and auxiliary supervision. Image decoding is optional. Choices can overlap; for example, a jointly trained model can also support forward prediction under supplied actions. Table 1 compares the main configurations of eight related systems.

of a frozen DINOv2 encoder, using a predictor of about 19M parameters trained on 18,500 trajectories [19]. PILOT initializes the cross-entropy method from a learned policy and reports gains over executing the same policy on long Push-T tasks [12]. AD-WM finds that factual prediction error does not follow closed-loop planning success [15], and SufficientPlan reports competitive control with much smaller search budgets than are usually used [10]. We test the same questions with a model trained from scratch whose every component, encoder included, totals about a fourteenth of DINO-WM's predictor alone, and in an environment where the planner's predictions can be compared with the simulator's at every step. Small settings make these choices inexpensive to examine, but cannot establish that the same mechanisms explain results in large robot models.

System	Predicts	Backbone	Action decoding	Future used for
GR-1 [17]	future frames (image patches)	GPT-style causal transformer, video-pretrained on Ego4D clips	linear head on an `[ACT]` token	training signal
UWM [20]	video and actions	one diffusion transformer, independent timesteps per modality	action diffusion	policy, forward or inverse dynamics, video generation
WorldVLA [3]	next frame (VQ tokens)	autoregressive multimodal LLM (Chameleon init.)	discrete action tokens	training signal
Genie Envisioner [11]	video chunks	video diffusion transformer (2B)	flow-matching action module on video latents	action decoding; neural simulator
Cosmos Policy [8]	future states and values as latent frames	Cosmos-Predict2 video diffusion (2B)	actions injected as latent frames	best-of-N planning
V-JEPA 2-AC [1]	future representations	frozen V-JEPA 2 encoder + 300M action-conditioned predictor	none: actions come from the planner	CEM model-predictive control
DreamGen [7]	robot videos	fine-tuned video generation model	inverse dynamics or latent actions label the videos	synthetic training data
LingBot-VA [9]	video latents, autoregressively	dual-stream diffusion transformer (5.3B), KV cache	inverse dynamics from the predicted future	action decoding; memory
pico-WAM (ours)	physical state, pixels, latents	3-layer CNN + GRU + MLPs (0.75–1.3M)	MLP, diffusion or flow head	training signal; analysis

Table 1: Comparison of eight open-source predictive robot-learning systems and pico-WAM. Entries describe the main configuration reported in each paper. The table includes joint policy models, forward models used by planners, and video generators used to create training data. Their prediction targets and deployment roles differ, so model size alone is not a comparison of capability.

3 Model and experimental setup

Architecture. The observation comprises a 64×64 RGB image o_t, an instruction ℓ, and, when available, a low-dimensional state s_t: six numbers for the point mass and eight for Push-T. A three-layer CNN, a text GRU, and a state MLP encode these inputs. The image encoder combines 16 spatial-softmax keypoints with pooled image features. A fusion MLP followed by layer normalization produces $z_t \in \mathbb{R}^{128}$ (Figure 3).

The policy maps z_t to a two-dimensional action. The implementation supports MLP, diffusion, and flow heads, optionally predicting a chunk of the next K actions. The point-mass experiments use an MLP head with one hidden layer of 128 units and $K = 1$. On Push-T, the final configuration uses three hidden layers of 512 units and predicts $K = 8$ actions, executing the first four before predicting again, as in Diffusion Policy [4]. The transition model is a residual MLP, $f_\theta(z, a) = \mathrm{LN}(z + \mathrm{MLP}[z, a])$. Starting from

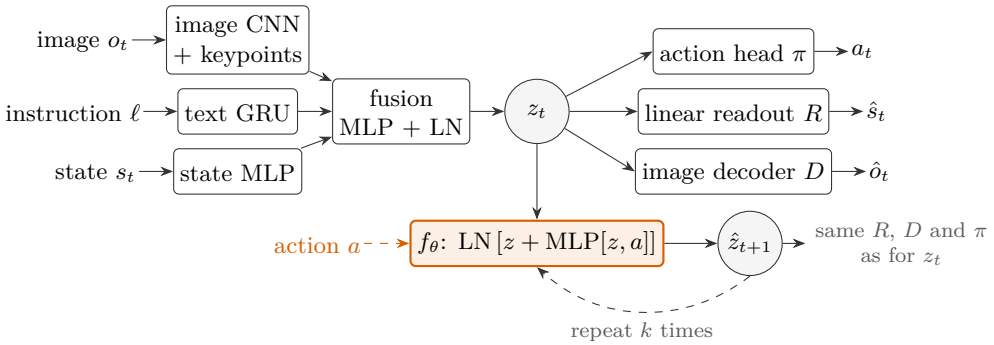


Figure 3: pico-WAM architecture. Image, instruction, and state encoders produce features that are fused into z_t. The action head, linear state readout R, and image decoder D use this representation. The residual transition model f_θ predicts the next latent from (z_t, a_t); the same readouts decode subsequent latents. Layer normalization controls latent scale but does not ensure that predicted and encoded latents have identical distributions. Figure 21 reports parameter counts.

$\hat{z}_t = z_t$, it predicts successive latents through $\hat{z}_{t+k+1} = f_\theta(\hat{z}_{t+k}, a_{t+k})$. A linear readout R predicts the normalized physical state, and a decoder D predicts an image residual relative to the training-set mean image. Both readouts apply to encoded and predicted latents. Because R is trained with state labels, its accuracy measures supervised state reconstruction, not independently discovered physical structure.

Training objective. Each sample is a window of H transitions from one episode. The model encodes the initial observation and predicts the next H latents using the recorded actions (Figure 4). Training minimizes

$$\mathcal{L} = \mathcal{L}_{\text{act}} + w_s \mathcal{L}_{\text{state}} + w_i \mathcal{L}_{\text{image}} + w_z \mathcal{L}_{\text{latent}}. \tag{1}$$

The state loss adds current-state reconstruction MSE to mean future-state MSE across the H predictions; the image loss has the same form. The latent loss averages MSE between predicted latents and stop-gradient encodings of the corresponding future observations. All MSE terms average over output coordinates. State targets use training-set means and standard deviations. The default weights are $w_s = 0.5$ and $w_i = w_z = 1$, with $H = 5$.

The action loss is evaluated only on expert-labeled transitions. Removing the transition model, the readouts and their losses leaves mini-VLA [14], the small vision-language-action model whose encoders pico-WAM extends, and we use it as the behavioral-cloning comparator. It shares pico-WAM's image encoder and action head, so the two models differ only in the world model and its losses, and it is trained from scratch on the same demonstrations. At deployment, both policies encode the current observation and emit an action; neither uses imagined states to select that action.

Point-mass navigation. The agent moves toward one of two colored goals selected by the instruction (Figure 5). Position p_t lies in the unit square, action $a_t \in [-1, 1]^2$ applies a force, and physical velocity follows

$$v_{t+1} = 0.8 v_t + 0.01 a_t, \qquad p_{t+1} = \text{clip}(p_t + v_{t+1}, 0, 1).$$

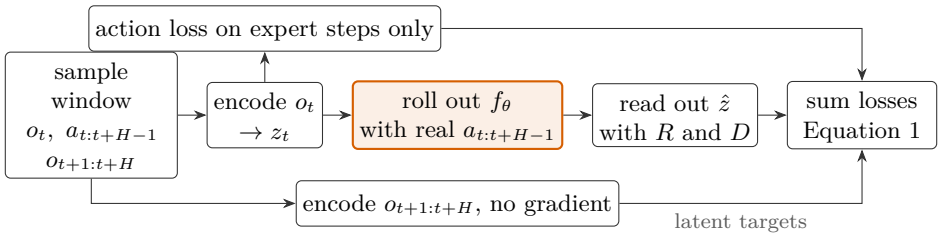


Figure 4: Training on a window of H transitions. The initial observation is encoded with gradients, and the transition model is rolled forward using recorded actions. State and image readouts are trained against current and future targets. Future observations are encoded to form stop-gradient targets for latent consistency; they are not inputs to the rollout. The action loss uses only expert-labeled steps, whereas prediction losses use all sampled transitions.

The observed state is $(p_t, v_t/0.05, g)$, where g is the selected goal position. Success requires distance to g below 0.05 and normalized speed below 0.2. A proportional–derivative controller supplies expert actions. When no random burst is active, the collector starts one with probability 0.3; its duration is sampled uniformly from one to three steps. Each burst action is sampled uniformly from $[-1, 1]^2$. Thus 0.3 is the burst-start probability, not the fraction of random transitions.

In the state-input condition, the selected goal coordinates already identify the target, so the policy need not use language to select it. In the pixel-input condition, the state-encoder output is set to zero, but the WAM still receives state-prediction targets during training. Velocity is not rendered: one image can correspond to different velocities. We therefore distinguish pixel inputs from training without state supervision.

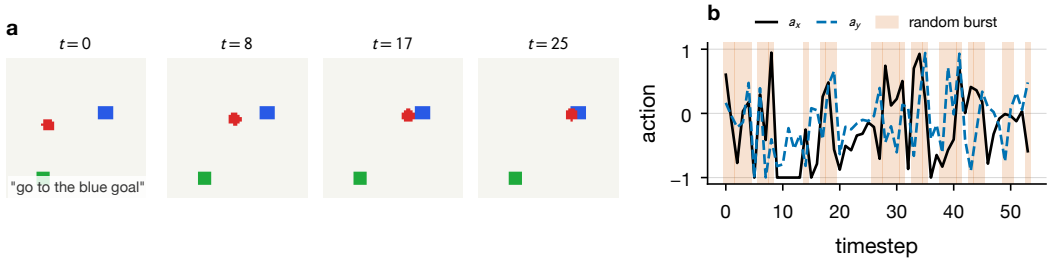


Figure 5: Navigation environment and data collection. (a) Actual frames from an expert episode. The red agent must reach the instructed goal and slow down; the other colored square is a distractor. Images show positions but not velocity. (b) Actions from a collection episode, with random-action bursts shaded. When no burst is active, one starts with probability 0.3 and lasts one to three steps. These transitions contribute to prediction losses but are excluded from action regression.

Push-T. The agent is a disk of radius 15 px that tracks a target point with a proportional–derivative controller in a 512×512 workspace. It pushes a T-shaped block, a 120×30 bar over a 30×90 stem, onto a fixed target pose through frictionless contact (Figure 6). Physics runs at 100 Hz and control at 10 Hz with the original constants [4]. We make two changes. An action $a_t \in [-1, 1]^2$ moves the controller's target up to 100 px from the agent instead of placing it at an absolute position, so a random action is a local nudge and a constant action pushes in one direction. The block's velocity is also reset

at every control step; under the original zero damping it carries over for only one 10 ms substep. With this change, the observed state determines the next one: re-simulating from a recorded state reproduces the next state exactly. The eight-dimensional state holds the agent's position and velocity and the block's position and orientation, $\cos\theta$ and $\sin\theta$, in the unit square. Images are drawn like the dataset's frames, on a 512×512 canvas resized to 96×96 and then to 64×64. Success requires the block to cover more than 95% of the target; we also report the best coverage reached in an episode, the benchmark's usual score.

The training data are the 206 human demonstrations of LeRobot's Push-T dataset, 25,650 frames at 10 Hz [2]. The dataset records the agent's position and an absolute target position, but not the block. We read the block's pose from the T's corners in the dataset's keypoint version, recover the agent's velocity by replaying the controller from rest, which reproduces every recorded position to within 10^{-4} px, and convert each target into a relative action; 29 targets lie more than 100 px away on an axis and are clipped. Dropping each episode's last frame leaves 25,444 transitions. From a recorded state, our simulator predicts the next recorded block position to within 0.53 px in 99% of steps, while a moving block travels a median 3.7 px per step. The demonstrations never reach the success threshold: their best coverage averages 84.8%. For data-coverage experiments we also collect 200 episodes per run with a scripted expert that chooses each push by simulating about a hundred candidates and walks around the block to make it. Random-action bursts start with probability p, as in the point-mass task, and draw each action from the whole action range. The expert succeeds in 100% of 30 test starts without bursts, but in only 17% at $p = 0.1$: a random action can undo the final placement.

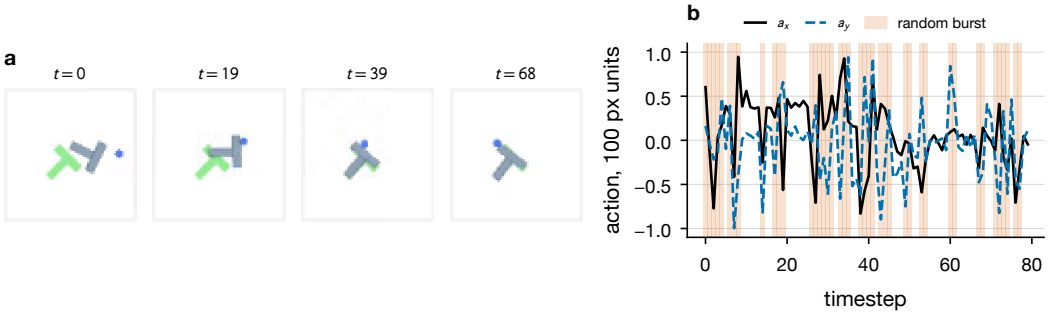


Figure 6: Push-T. (a) Frames of an unseen human demonstration from LeRobot's dataset, as the model sees them. The blue agent must push the grey T onto the green target. (b) Actions of a scripted-expert episode with random-action bursts shaded, at burst-start probability 0.3. An action moves the controller's target by up to 100 px from the agent on each axis.

Planning with the world model. On Push-T we also let the WAM act by planning with its own dynamics. At each step the planner encodes o_t, imagines the policy's own next 12 actions, and samples 256 candidate sequences around them with Gaussian noise of standard deviation 0.1, always including the policy's plan itself. Each candidate is rolled out with f_θ, and R reads the predicted block pose at every step. A candidate's cost is the mean distance in pixels between the T's corners and the target's corners, averaged over the 12 steps with weights rising from 0.5 to 1.5, plus 0.05 times the distance between the agent and the block. The first action of the cheapest candidate is executed, and the

next step's search starts again from the policy. Our first planner instead started each search from the previous step's plan, shifted by one step, and kept that plan unless a candidate scored better; we call it the warm-start planner and compare the two below. The planner never calls the simulator. Its settings were fixed on 20 development starts, which are never used for testing. Figure 7 shows one planning step of the final model.

Every 10 steps of a planning episode, we also check the chosen plan: from the same state, the simulator executes the whole 12-step plan open loop. Distances between block poses are mean distances between the T's eight corners, in pixels, as in the planner's cost. The check reports the distance between the imagined and the simulated final pose, which we call plan error, and how much closer to the target the model expected the plan to bring the block, against how much closer it really came.

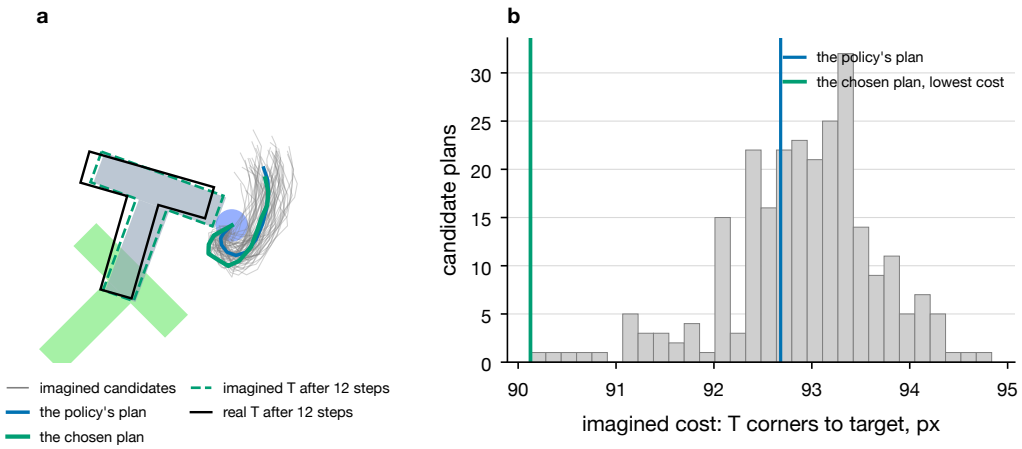


Figure 7: One planning step. (a) From the current state, the world model imagines 256 action sequences of 12 steps around the policy's own plan; grey lines are the agent paths it imagines for every fourth candidate. The chosen plan has the lowest imagined cost. Its imagined final T pose, dashed, is close to the pose the simulator produces for the same actions, solid. Only the first action is executed before the search starts again. (b) Imagined costs of all 256 candidates: the mean distance between the T's corners and the target's over the 12 steps, plus a small pull of the agent toward the T.

Evaluation protocol. Each configuration uses three independent split and initialization seeds and a 90/10 episode split, and trains for 15 epochs unless stated otherwise; the final Push-T configuration trains for 100. In the point-mass task each seed also collects its own 400 episodes; on Push-T the human demonstrations are the same in every run, and scripted data are collected per seed. Closed-loop evaluation uses 100 unseen starts per run, of 80 steps for the point mass and 300 for Push-T. Prediction is evaluated on unseen episodes by encoding each valid starting observation once and rolling forward without observing later frames. Factual evaluation uses recorded actions. Counterfactual evaluation samples eight random action sequences at starts spaced five steps apart and compares each prediction with the simulator. Random actions span the full action range in the point-mass task and up to 40 px on each axis on Push-T, about twice a typical human action.

State error is mean squared error after dividing each coordinate's residual by its training-set standard deviation, written nMSE. Factual error averages all state coordi-

nates. Point-mass counterfactual error averages position and velocity, the coordinates that actions move; on Push-T it averages all eight, and we also report the four block coordinates separately. Image error is pixel MSE on RGB values scaled to $[0, 1]$. Factual persistence repeats the initial state or image. The counterfactual reference instead runs the true simulator with zero actions from the same initial state.

The main studies report population standard deviations over three runs; the five-run supervision pilots report sample standard deviations. These are measures of run-to-run variation, not confidence intervals. Overlapping prediction windows are correlated. Exploration conditions also change state distributions and normalization statistics, so their errors describe the resulting datasets rather than an isolated effect of action diversity.

4 Point-mass results

We first evaluate the forward model, then examine whether auxiliary prediction improves the policy. Table 2 summarizes the main comparisons. It also records decoder and diffusion-schedule changes made during development; those observations are distinguished from the repeated experimental comparisons.

State and image prediction. At horizon 15, factual state NMSE is 0.0071 ± 0.0004, compared with 0.5824 for persistence. At horizon 10, counterfactual error is 0.0072, compared with 0.388 for the zero-action simulator reference (Figure 8). The model therefore predicts motion substantially better than these references. The comparison does not establish an advantage over a fitted state-space model.

Image prediction is less successful. Pixel MSE exceeds frame copying at every evaluated horizon, despite accurate state readouts. In an unseen example episode, the decoder retains the agent and selected goal but fails to reconstruct the distractor. This example illustrates a reconstruction failure; it does not establish why the distractor is lost. The large static background also makes copying a strong pixel baseline.

Prediction under alternative actions. Expert-only training gives lower long-horizon factual error but a counterfactual error of 0.112 at horizon 10, versus 0.0072 with burst-start probability 0.3 (Figure 9). Accurate prediction along expert trajectories therefore does not imply equally accurate prediction under other actions. A network may exploit correlations between states and expert actions, as discussed in work on causal confusion [5]; this experiment does not directly measure whether the network ignores its action input.

A separate five-run diagnostic fits an affine state-and-action predictor, with the known position-clipping rule, to deterministic expert trajectories. Its ten-step counterfactual position MSE is approximately 4.3×10^{-16}. Expert data therefore identifies the transition within this restricted model class. The neural result reflects the interaction between data and model fitting, rather than an unavoidable lack of information in the demonstrations. These raw position errors are not directly comparable with neural NMSE.

Sustained-action prediction. We also evaluate 15 consecutive steps of constant force, longer than the one-to-three-step random bursts in training. Figure 10 compares pre-

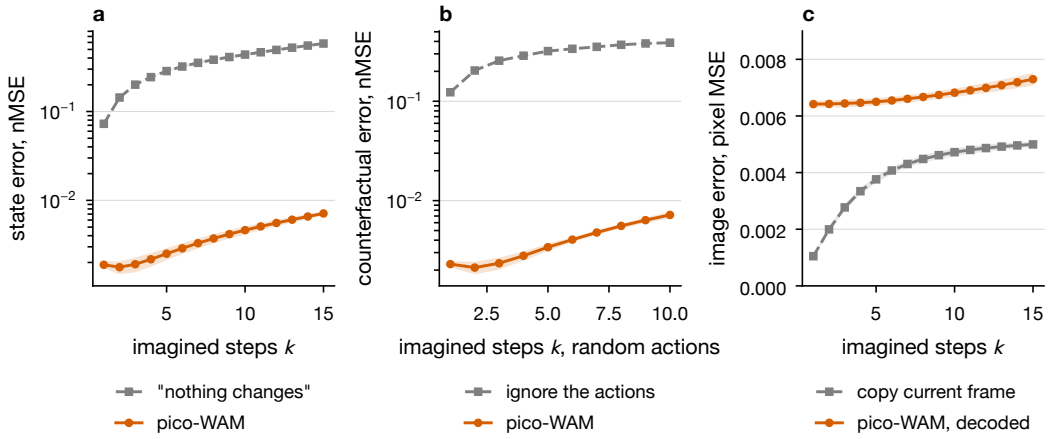


Figure 8: Prediction error versus rollout horizon. (a) Factual normalized state MSE compared with persistence. (b) Counterfactual normalized MSE under random actions, compared with a true-simulator rollout that applies zero actions. (c) Pixel MSE compared with repeating the initial frame. Lines show means and bands population standard deviations across three runs, each with 40 unseen episodes. State prediction improves on these references; image prediction does not.

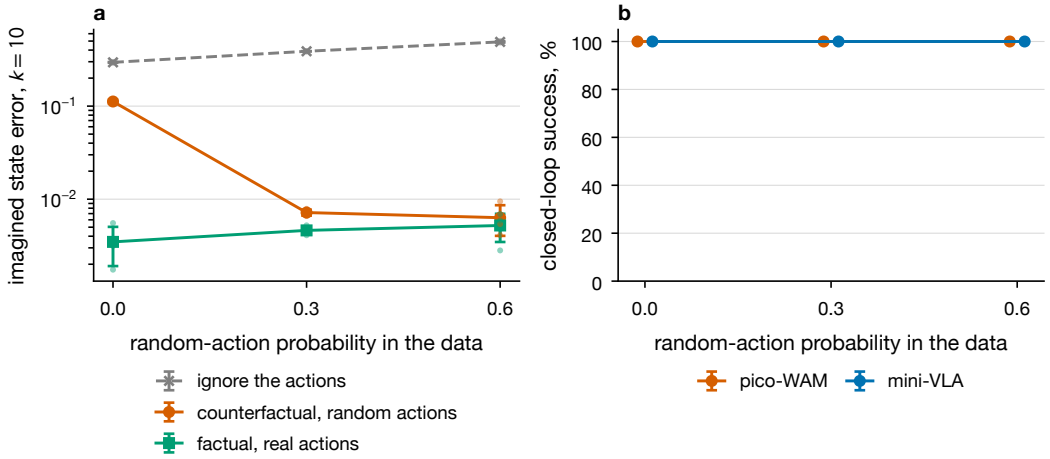


Figure 9: Effect of the data-collection policy. (a) Ten-step factual and counterfactual errors at different burst-start probabilities. Expert-only data gives lower factual error but higher counterfactual error. Grey marks the zero-action simulator reference. (b) Closed-loop success remains high across these conditions; only expert-labeled steps contribute to imitation. Dots denote runs and bars population standard deviations over three runs. The horizontal setting controls burst initiation, not the measured fraction of random steps; trajectory distributions also change.

Component	Comparison	Observation	Interpretation
Training data	expert only vs. burst-start probability 0.3	counterfactual error at k=10 falls from 0.112 to 0.0072, even though factual error is lower with expert-only data (Fig. 9)	lower counterfactual error
Imagination horizon H	1, 5, 10 steps	10-step counterfactual error 0.0139, 0.0072, 0.0057; one-step error 0.0019, 0.0019, 0.0030	short-/long-horizon trade-off
Latent consistency	on vs. off	15-step error 0.0071 ± 0.0004 vs. 0.0096 ± 0.0024; one-step error slightly better without it	lower long-horizon error and spread
Image loss	on vs. off (state input)	15-step state error 0.0071 vs. 0.0074	similar observed means
Decoder output	sigmoid vs. residual on the mean image	sigmoid training stalled; the residual decoder learns, but remains worse than frame copying (Fig. 8c)	limited image fidelity
Visual representation	keypoints with vs. without state input	with state input the keypoints barely move; without it they track the agent, $R^2 \geq 0.998$	stronger localization without state input
Diffusion head	linear schedule ($\bar{\alpha}_T \approx 0.92$) vs. cosine ($\bar{\alpha}_T \approx 10^{-5}$)	the linear schedule never reaches noise, yet sampling starts from noise; fixed with a cosine schedule and the DDPM posterior	noise-schedule correction
Auxiliary supervision	BC, + current state, + future state (pixel input)	success 17.0%, 28.4%, 33.6%; current-state supervision gives the larger increment (Fig. 11c)	future prediction adds 5.2 points
Few demonstrations	25 to 400 episodes	WAM trails mini-VLA at 25 episodes (87% vs. 99.7%); matched updates close most of the gap	sensitive to update budget

Table 2: Experimental comparisons and implementation observations. State errors are normalized MSE, and policy success is measured on 100 unseen episodes per run. Main comparisons use three runs; the supervision pilot uses five. Decoder and diffusion-schedule rows describe development observations, not repeated ablations. Factual error averages six state coordinates; counterfactual error averages position and velocity only. Similar means do not establish equivalence.

dicted and simulated trajectories from one initial state. Several predictions underestimate displacement. Across unseen episode starts, mean endpoint distance is 0.103 ± 0.010 in unit-square coordinates, compared with 0.457 when every action is assigned the zero-force endpoint. The model captures much of the action-dependent displacement but retains appreciable long-horizon error.

Visual representations. With state inputs, the 16 spatial-softmax keypoints vary little and remain near the image center. Removing state inputs produces keypoints that track the agent and goals; the best-keypoint tracking score is $R^2 \geq 0.998$ in each run. This is consistent with greater reliance on visual localization when positions are not supplied directly. It is not a controlled comparison between spatial-softmax and global-pooling encoders.

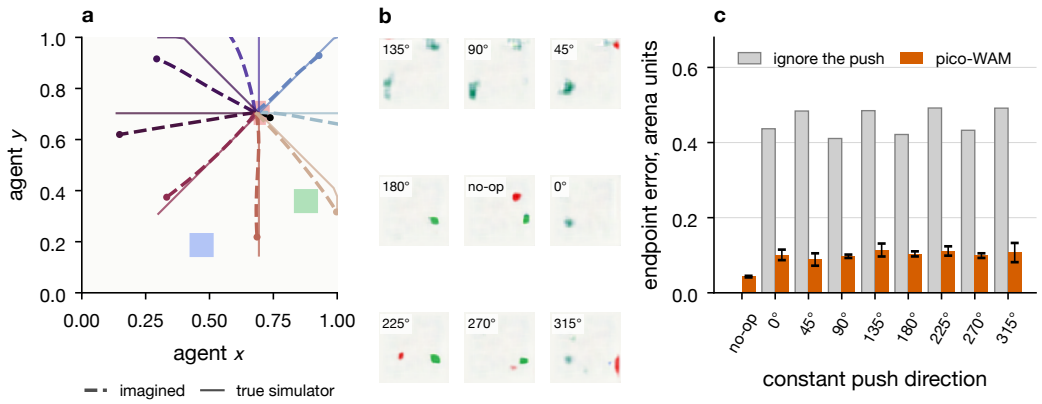


Figure 10: Predictions under sustained force. (a) Predicted (dashed) and simulated (solid) trajectories for 15 steps of constant force in eight directions, plus a zero-force condition. (b) Predicted final frames, arranged by force direction. (c) Endpoint distances averaged over the initial states of 40 unseen episodes per run; whiskers show population standard deviations across three runs. Grey bars assign the zero-force endpoint to every action sequence.

Policy performance. With state inputs and 400 episodes, both WAM and mini-VLA achieve 100% success. With 25 episodes and 15 epochs, WAM achieves 87.0%, compared with 99.7% for mini-VLA. Extending training to 225 epochs narrows this difference to 96.3% versus 98.7% (Figure 11a). Update budget is therefore an important factor in the low-data comparison.

With pixel inputs, success is $80.3 \pm 12.0\%$ for WAM and $36.0 \pm 19.8\%$ for mini-VLA (Figure 11b). WAM performs better in all three runs, but it also receives state targets that mini-VLA does not. In addition, the original WAM uses five-step training windows, while mini-VLA uses one-step windows. These windows exclude different episode-end samples.

To separate current-state supervision from future prediction, a five-run Explorer pilot uses 200 episodes per run, 15 epochs, and matched five-step starts, initialization, batches, and update counts. Success is 17.0% for cloning alone, 28.4% with current-state supervision, and 33.6% with current and future-state supervision (Figure 11c). The increments are 11.4 and 5.2 percentage points; the latter ranges from -2 to $+9$ points across runs. The pilot excludes image and latent-consistency losses. Its lower data budget and different objective prevent attributing the original 44.3-point gap to the same proportions.

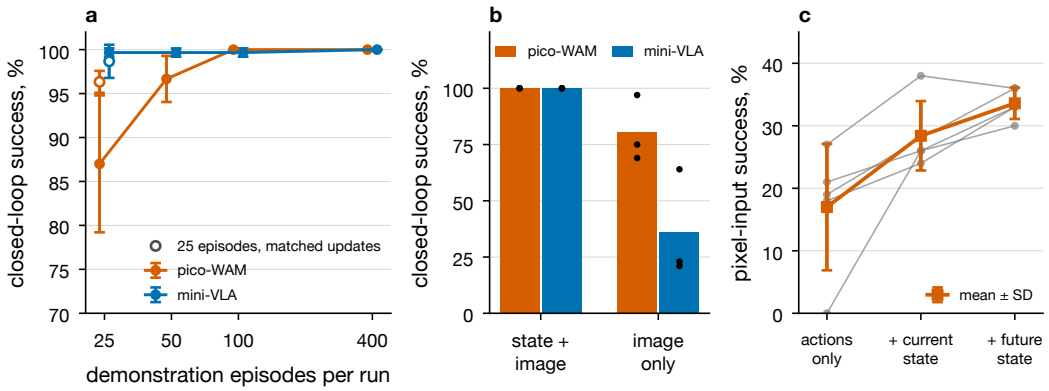


Figure 11: Policy success under different data and supervision conditions. (a) Success versus collected episode count after 15 epochs. Open markers use 225 epochs on 25 episodes to approximately match the standard update budget. (b) State-input and pixel-input policies trained on 400 episodes, with dots for individual runs. (c) Matched pixel-input pilot on 200 episodes per run: cloning, current-state supervision, and current plus five-step future-state supervision. Grey lines connect five runs; orange shows mean and sample SD. Panels (a,b) use three runs and the full WAM objective; panel (c) excludes image and latent-consistency losses.

5 Push-T results

We first evaluate the world model against simple references, including a linear model fitted to the same data, then compare reactive policies with planning.

Prediction along human actions. On unseen demonstrations, pico-WAM predicts the state 15 steps ahead with nMSE 0.0178, against 0.7772 for persistence (Figure 12a). A linear model of the state change, fitted by least squares on the same training transitions from the state and action, is more accurate one step ahead, 0.0007 against 0.0018, because the agent follows a linear controller. It is less accurate after 15 steps, 0.0665, once contact has moved the block. Decoded images are worse than copying the current frame one step ahead, 0.00098 against 0.00041, and better after 15 steps, 0.00145 against 0.00305, once the scene has changed.

Prediction under other actions. Under random actions, the counterfactual error after 10 steps is 0.0295, against 0.364 for the zero-action reference and 0.0171 for the linear model (Figure 12b). The average hides a difference between the two objects. On the block's four coordinates alone, the WAM's error is 0.0344, higher than the 0.0253 obtained by assuming the block stays where the zero-action simulator leaves it; the linear model reaches 0.0326. Along recorded human actions the block error after 10 steps is 0.0162. The world model has learned how the block moves under the pushes people made, more than how it responds to others. Sustained pushes show the same pattern. After 15 steps of constant pushing in each of eight directions, the predicted agent endpoint is off by 0.069 of the workspace width, against 0.406 if the push is ignored, while the predicted block endpoint is off by 0.057, against 0.023 if the push is ignored.

Policies and planning. Acting reactively, the WAM reaches a best coverage of 14.0% ± 3.2, below mini-VLA's 24.2% ± 6.1. Planning with the same world model raises it

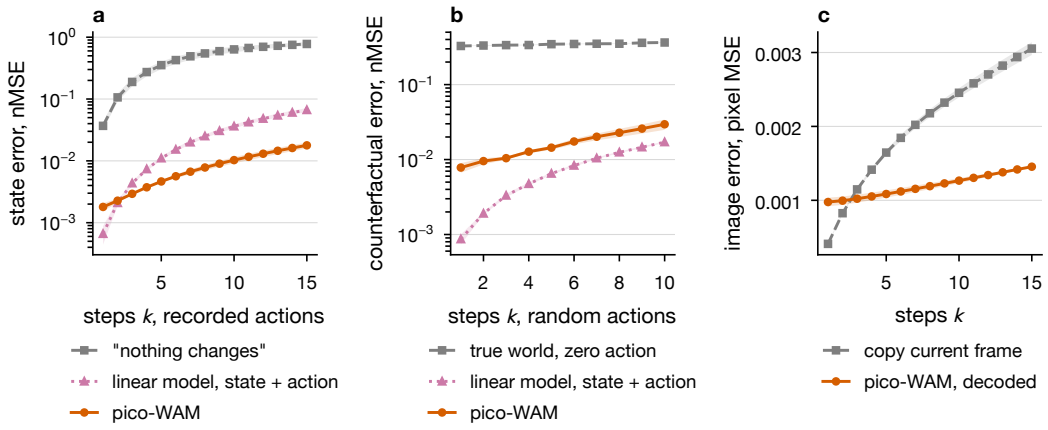


Figure 12: Push-T prediction error versus rollout horizon. (a) Factual nMSE along unseen human demonstrations, against persistence and a linear model of state and action fitted to the same data. (b) Counterfactual nMSE under random actions within 40 px, against the zero-action simulator and the linear model. (c) Pixel MSE against repeating the initial frame. Lines show means and bands population standard deviations across three runs, each with 21 unseen episodes.

to 37.0% ± 5.6, above mini-VLA in each of the three runs. No policy succeeds, and no demonstration does either: the demonstrations' best coverage averages 84.8%.

Search budget. Figure 13 gives the planner more search on the same trained models, all with the warm-start planner of the study base: noise 0.3 instead of 0.1, three cross-entropy rounds with noise 0.3, or three rounds that start from standing still instead of the policy's plan. Coverage falls each time, from 33.8% to 23.0%, 18.2% and 9.6%. The plan checks show why. The improvement the model expects from its chosen plan grows with the search, from 33.3 px to 82.1 px with three rounds, while the simulated improvement of the same plans stays near zero: −1.5, −1.3, −1.9 and −0.1 px. Plan error grows from 47.6 to 104.5 px. A stronger search finds the actions whose outcome the model gets most wrong. This agrees with PILOT's use of a policy prior [12] and with SufficientPlan's finding that small budgets suffice [10]; the simulator lets us measure the cause directly.

Which data teaches the block's response. The zero-action reference depends on the test episodes, so we compare datasets by the ratio of the WAM's block error under random actions to that reference on the same starts; below one, the model beats assuming that the block stays put (Figure 14). Trained on human demonstrations, the ratio is 1.36. Trained on the scripted expert it is 0.53 without random bursts, and 0.43 and 0.66 with bursts at $p = 0.1$ and 0.3; human demonstrations plus scripted episodes with bursts at $p = 0.3$ give 0.45. The scripted expert pushes from many contact points because it searches over candidate pushes, and bursts add further actions. The policies do not improve with it. On scripted data mini-VLA reaches 11.3% to 13.8%, against 24.2% on human demonstrations, and planning reaches 24.3% to 30.6%, against 37.0%. We did not test why the scripted expert is harder to imitate; one candidate is that its choices depend on a lookahead the policy cannot observe. Planning needs both a world model

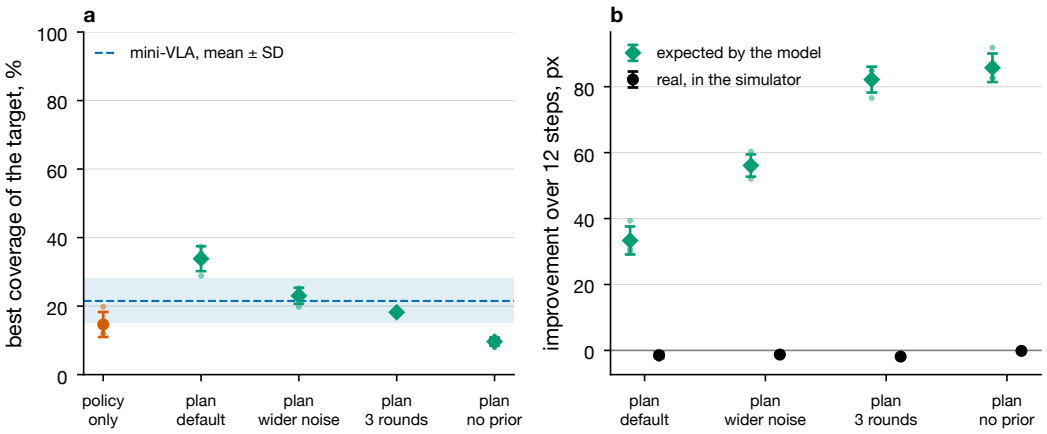


Figure 13: More search exploits the world model's errors. (a) Best coverage of the reactive WAM and of four planners using the same models: the default, noise 0.3, three cross-entropy rounds with noise 0.3, and three rounds that start from standing still instead of the policy's plan. The blue band is mini-VLA, mean ± SD. (b) Every 10 steps, the chosen 12-step plan is executed open loop in the simulator from the same state: how much closer to the target the model expected the block to come, and how much closer it came. Dots are runs, bars population standard deviations over three runs.

that knows the block's response and a policy whose plans are worth searching around, which led us to branch the human demonstrations instead (below).

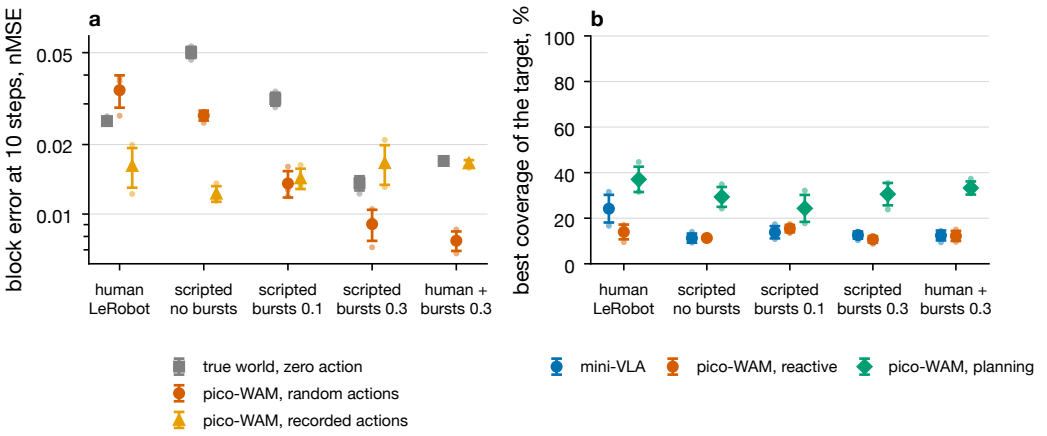


Figure 14: Training data and the block's response. (a) Block nMSE after 10 steps, under random actions and along recorded actions, with the zero-action simulator on the same starts in grey, for each training dataset. Each dataset is tested on its own unseen episodes. (b) Best coverage of mini-VLA, the reactive WAM and the planning WAM. Scripted data comes from the expert with random bursts starting with probability p. Dots are runs, bars population standard deviations over three runs.

Objective and model variants. The upper group of Figure 15 varies one setting of the study base. Planning depends on latent consistency: without it, coverage falls to 23.7%, although counterfactual error falls slightly. The image loss is not needed for

planning, 38.1% without it. A one-step training horizon lowers planning to 27.8%, and ten steps give 33.9%. Fewer demonstrations lower every policy: with 100 and 50 of the 206, planning reaches 24.5% and 19.0%. From pixels alone, without state input, coverage drops to about 10% for every policy, and the world model's state error after 15 steps rises to 0.0838. A diffusion head lowers every policy at this training length, to 14.2% for mini-VLA. Doubling the latent width to 256 does not help: planning reaches 35.2%, and the block's counterfactual error rises to 1.96 times the reference.

Predicting chunks of eight actions and executing four, as in Diffusion Policy, raises the reactive WAM to 23.5% and mini-VLA to 40.2%, with the first successes of the study: 5 and 4 in 300 episodes. Planning with the chunked model reaches 39.2%. The lower group varies this chunked model. Predicting 16 actions and executing eight gives 32.5% for mini-VLA, and averaging overlapping chunks gives 39.0%; neither improves on $K = 8$. Halving the latent width to 64 lowers planning to 36.3%. Replacing the keypoint encoder with a plain CNN and global average pooling gives 44.7% for mini-VLA and 43.6% for planning, within the spread of the runs. Without the image decoder, planning reaches 38.0%, against 39.2% with it, at about a tenth of the training cost (Section 6). At 15 epochs, neither a narrower nor a wider latent improves on 128.

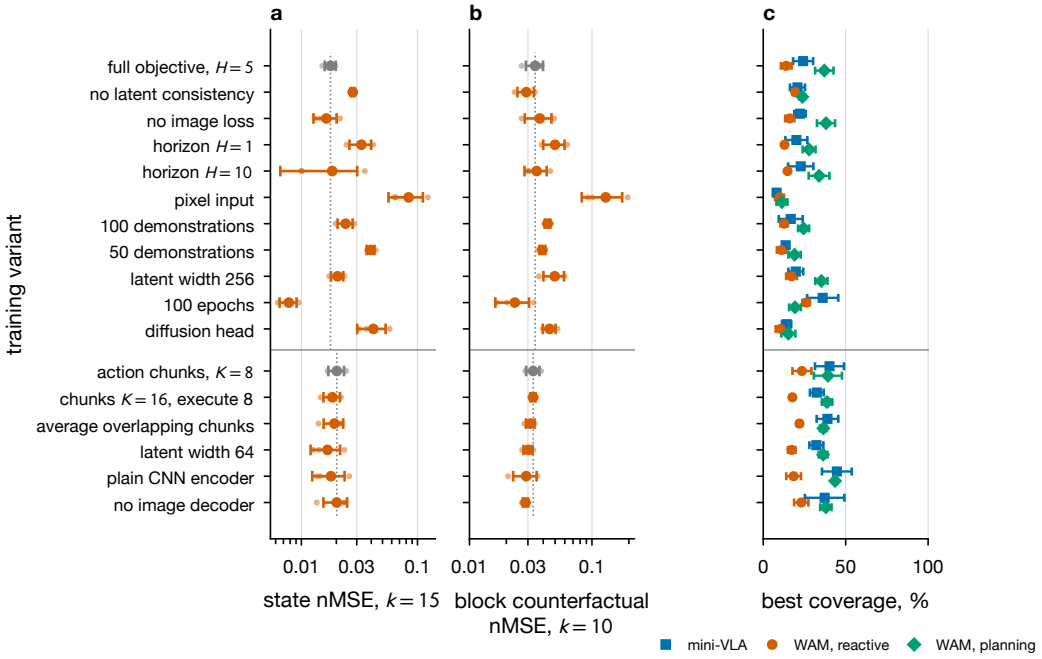


Figure 15: Push-T variants. Upper group: each row changes one setting of the study base, grey, which predicts one action per step. Lower group: each row changes one setting of the model with action chunks, grey. (a) State nMSE after 15 steps along recorded actions. (b) Counterfactual nMSE of the block after 10 random-action steps; dotted lines mark each group's reference row. (c) Best coverage of the target for mini-VLA, the reactive WAM and the planning WAM. Dots are runs, bars population standard deviations over three runs, 100 unseen starts per run.

Training longer, and why planning got worse. Training the chunked models for 100 epochs instead of 15 raises mini-VLA to 68.0% ± 2.3 with 62 successes in 300 episodes,

and the reactive WAM to 46.4% with 9 (Figure 16). The world model also improves: its state error after 15 steps falls from 0.0202 to 0.0112. Planning, however, falls from 39.2% to 20.2%, and 100 epochs without chunks show the same drop, 19.3%. Both runs used the warm-start planner. On development starts, which are never used for testing, its chosen plans with the 100-epoch model expected to bring the T 74 px closer to the target in 12 steps while the simulator moved it 7 px further away. Because the warm-start planner keeps the previous plan unless a candidate scores better, an episode accumulates hundreds of rounds of search: in effect, the stronger search of Figure 13. Restarting each search from the policy's plan prevents this accumulation. On the same 15-epoch chunked models, the restarting planner reaches 44.0% with 4 successes and a plan error of 37.2 px, against 42.8% with 0 successes and 48.7 px for the warm start.

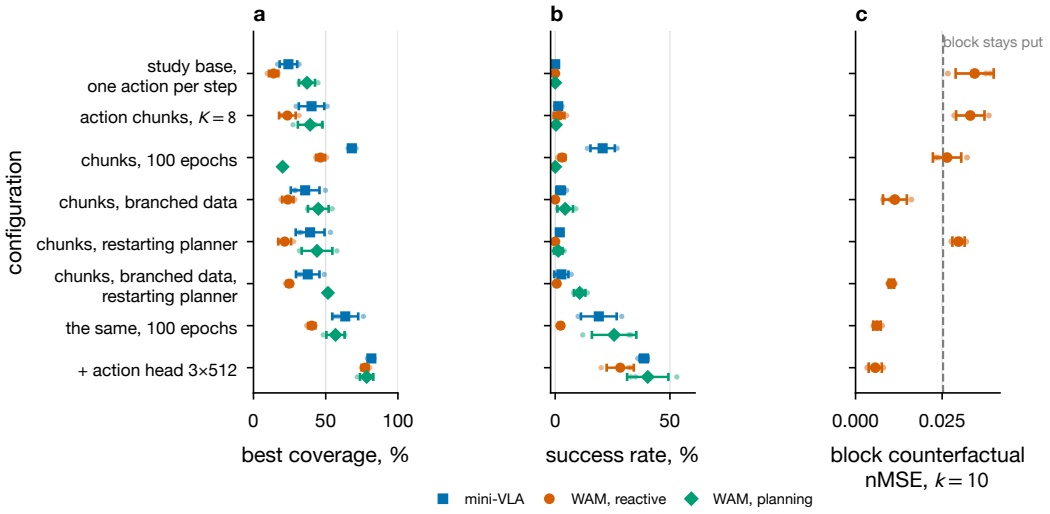


Figure 16: Changes that make Push-T succeed. Below the study base, each row adds the changes it names to the model with action chunks, which predicts eight actions and executes four; the seventh row trains the row above it for 100 epochs instead of 15, and the last row also gives both policies an action head with three hidden layers of 512. Planning uses the warm-start planner in the first four rows and the restarting planner in the others. Branched data reaches only the world model, so mini-VLA's differences between those rows are run-to-run variation. (a) Best coverage of the target. (b) Success rate, coverage above 95%. (c) Block counterfactual nMSE after 10 random-action steps; the dashed line is the zero-action simulator, which assumes the block stays put. Dots are runs, bars population standard deviations over three runs, 100 unseen starts per run.

Branching the demonstrations. The data comparison suggests that the world model needs to see the block respond to pushes people did not make, from states people did visit. We therefore branch the human demonstrations with the simulator. From every fourth state of each training demonstration, we replay the next eight human actions with Gaussian noise of standard deviation 0.3 added and record the outcome. This adds 6,013 branches, or 48,104 transitions, in 34% of which the block moves (Figure 17). Branches are marked as non-expert, so only the world model learns from them, and each stays on its source demonstration's side of the train/test split. Mini-VLA does not train on them: their actions are perturbed demonstrations, and imitating them would

teach the noise. The simulator is used only to build this data before training, never during planning. With branches, the block's counterfactual error falls from 1.31 to 0.45 times the zero-action reference, and planning with the warm-start planner reaches 44.9% with 13 successes. With branches and the restarting planner together, planning reaches 51.5% ± 1.4 with 32 successes, against 37.6% and 8 for mini-VLA trained in the same runs, and plan error falls to 31.9 px.

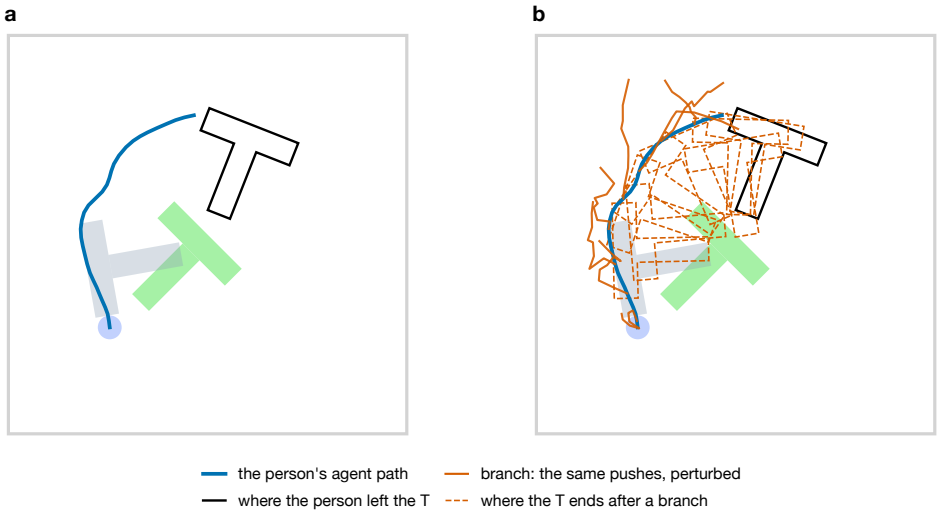


Figure 17: Branching a human demonstration. (a) A 32-step stretch of one demonstration: the agent's path and where the person left the T. (b) The same stretch with the branches started along it. Each branch replays the person's next eight actions with Gaussian noise of standard deviation 0.3 and lets the simulator say where the agent and the T end up. Only the world model learns from branches.

Training for 100 epochs. Training the model with branches and the restarting planner for 100 epochs, the seventh row of Figure 16, lets planning reach 56.8% ± 6.3 with 77 successes, against 63.5% and 57 for mini-VLA. On the same models, the warm-start planner reaches only 37.4% with 11 successes and a plan error of 40.2 against 32.5 px: the drift persists at 100 epochs, and restarting the search removes most of its cost. The reactive WAM stays far behind, at 40.3% with 7 successes, 23.2 points below mini-VLA.

The action head was the bottleneck. Up to this point, both policies used an action head with one hidden layer of 128 units. Figure 18 varies the head and the latent on top of the 100-epoch model. Three hidden layers of 512 raise every policy: mini-VLA to 81.6% with 116 successes, the reactive WAM to 77.2% with 85, and planning to 78.4% with 121. The reactive WAM now trails mini-VLA by 4.4 points instead of 23.2, so the small head, rather than the prediction losses sharing its latent, held the WAM's own policy back. Larger heads do not help further: three layers of 1024 give planning 77.9% and six layers of 512 give 72.0%. A flow-matching head with three layers of 512 lowers both reactive policies, mini-VLA to 65.5% with 12 successes, although planning still reaches 71.5%. A latent of 256 helps the one-layer head, with planning at 70.5% and 89 successes, but adds nothing to the 3×512 head, 77.2% with 108. The 3×512 head

has 599,568 parameters, so the WAM grows from 0.75M to 1.33M, or 0.96M without its image decoder, and mini-VLA to 0.82M. These head sizes were compared on the test starts, so the choice of 3×512 is itself fitted to them.

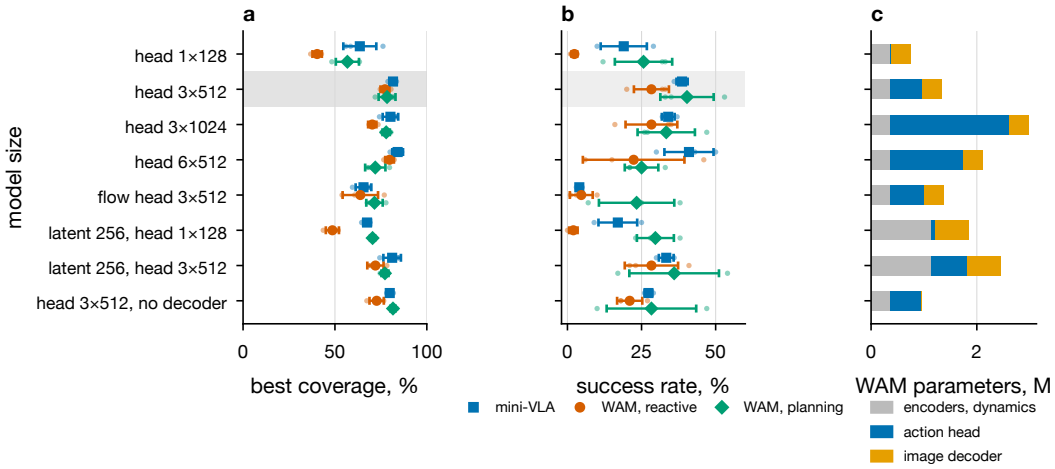


Figure 18: How big should pico-WAM be? Policy-head and latent sizes on top of the 100-epoch model with branches and the restarting planner; the shaded row is the final configuration. Both models get the same head in every row. (a) Best coverage of the target. (b) Success rate. (c) WAM parameters by part. A head of three hidden layers of 512 helps every policy; wider or deeper heads and a wider latent add parameters without improving planning. Dots are runs, bars population standard deviations over three runs, 100 unseen starts per run.

The final configuration. With the 3×512 head, planning and mini-VLA are close: planning succeeds in 121 of 300 unseen episodes and mini-VLA in 116, while mini-VLA reaches a slightly higher best coverage, 81.6% ± 2.0 against 78.4% ± 4.6, and has more successes in two of the three runs. The warm-start planner reaches 48.3% with 12. Without the image decoder, planning reaches 81.6% with 85 successes, and the run takes about a quarter of the time. The final world model predicts the state 15 steps ahead with nMSE 0.0094, against 0.4973 for persistence, and its block counterfactual error is 0.22 times the zero-action reference. Figure 19 follows one successful planning episode. Mini-VLA's configuration in the seventh row of Figure 16 is identical to the one in the chunked 100-epoch run, where it reached 68.0% with 62 successes instead of 63.5% with 57, which indicates the run-to-run variation of these numbers.

Pixels only. The clearest benefit of the WAM's training appears without state input (Figure 20). With the 3×512 head and 100 epochs, the reactive WAM reaches 56.4% with 43 successes and planning 53.5% with 46, while mini-VLA stays at 7.1% with 0. A single frame does not show the agent's velocity, and mini-VLA learns only from the action labels; the WAM also learns from state, image and latent targets, and from the branched data, which mini-VLA cannot use. The comparison therefore measures the whole WAM recipe rather than one loss. With the one-layer head at 15 epochs and without branches, both reached only about 10%.

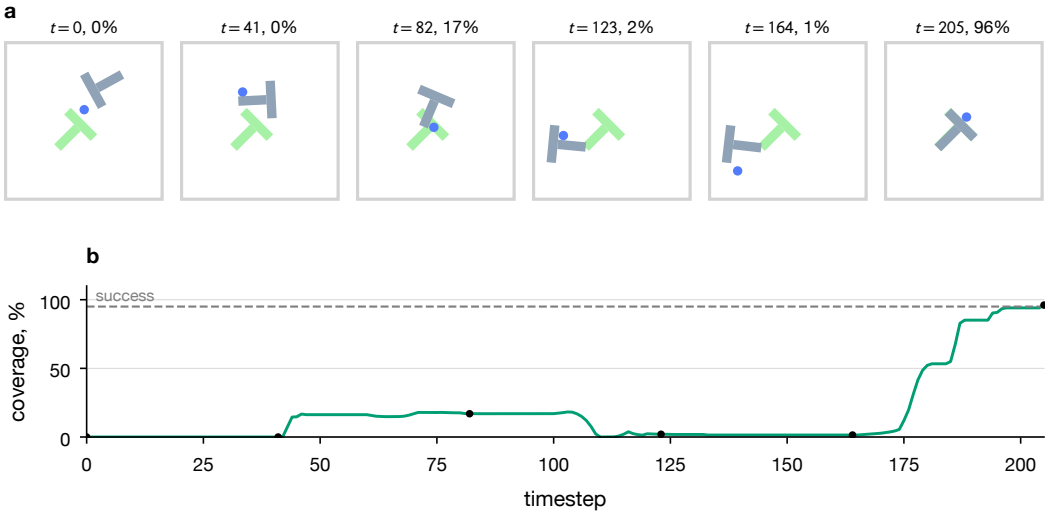


Figure 19: A successful planning episode of the final model on an unseen start. (a) The scene at six evenly spaced steps, with the coverage of the target. (b) Coverage over the episode; it succeeds when the T covers more than 95% of the target.

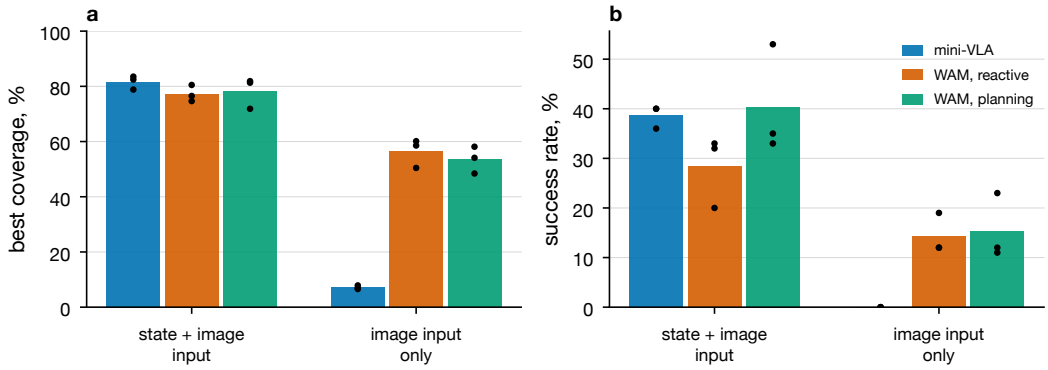


Figure 20: With images as the only input, the WAM still works. Final configuration with state and image input, and the same configuration with image and instruction input only. (a) Best coverage of the target. (b) Success rate. Dots are runs, bars means over three runs, 100 unseen starts per run.

Matched supervision pilot. We repeat the point-mass pilot on Push-T: pixel inputs, the human demonstrations, 15 epochs, five runs and matched windows, initialization, batches and updates. Best coverage is 6.3% ± 1.8 for cloning alone, 14.2% ± 4.7 with current-state supervision, and 11.7% ± 3.6 with current and future-state supervision. Current-state supervision adds 7.9 points; future-state prediction adds −2.5. As in the point-mass task, most of the measured gain comes from supervising the current state.

6 Computational cost

We profile the components using one CPU thread on an Apple-silicon laptop, with batch size one for inference and 64 for optimizer steps. Reported times are medians after warm-up. For the point-mass configuration, Figure 21 compares the full 747,739-parameter

model, a 371,928-parameter variant without image decoding, and the 238,674-parameter VLA.

Policy inference takes about 0.42 ms for both WAM and mini-VLA because both use the encoder and action head without calling the dynamics or decoders. A full WAM optimizer step takes 2.55 s, compared with 0.067 s for mini-VLA. Removing image decoding reduces the WAM step to 0.27 s, a reduction of about 90%. The image decoder contains 375,811 parameters, approximately half of the full model.

From an already encoded observation, ten latent transitions with state readouts take 0.28 ms; the corresponding rollout with image decoding takes 12.7 ms. These rollout times exclude the initial encoding. Tensor-indexed batch assembly takes 2.6 ms, compared with 4.5 ms for per-sample conversion and stacking. Image decoding dominates the measured cost; these CPU timings do not establish GPU performance.

On Push-T, the study-base model has 748,125 parameters and mini-VLA 238,802. The final configuration's action head, three hidden layers of 512, brings the WAM to 1,330,923 and mini-VLA to 821,600. On the same laptop thread, a final-model WAM optimizer step takes 3.18 s, or 0.29 s without the image decoder, against 73 ms for mini-VLA. A policy action takes 0.47 ms, and one planning step, 256 candidate sequences of 12 steps imagined in a batch, takes 36 ms, within Push-T's 100 ms control period. For comparison, DINO-WM reports 53 s for one cross-entropy plan of 10 iterations with 100 samples each [19]; its planner, horizon and hardware differ, so the comparison only indicates scale. Every Push-T run trained its three seeds in parallel on one cluster GPU. The final configuration's run, with 100 epochs on 73,548 transitions per seed for the WAM, took 4 h 11 min on one V100, and 1 h 03 min without the image decoder. The whole Push-T study, 37 completed runs, used about 90 GPU-hours.

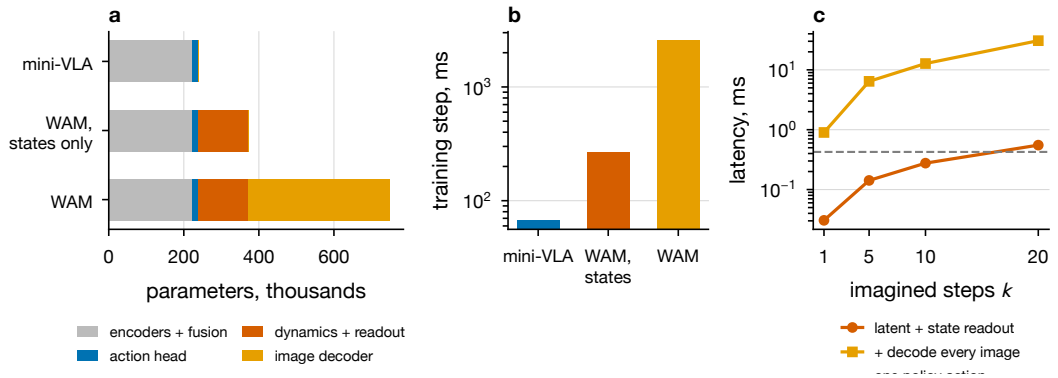


Figure 21: Parameter counts and CPU runtime. (a) Parameters by module for mini-VLA, WAM without image decoding, and full WAM. (b) Median optimizer-step time at batch size 64, on a log scale. (c) Median rollout latency from an already encoded observation, with state readouts or image decoding; the dashed line is one policy call, which includes encoding. Measurements use one CPU thread on an Apple-silicon laptop. Removing the decoder reduces WAM optimizer-step time by approximately 9.5×.

7 Limitations

The point-mass environment has affine dynamics away from position clipping. State inputs supply velocity and the selected goal, so these experiments do not establish lan-

guage grounding. On Push-T the state input gives the policy the block's exact pose, and the pixel-input WAM still receives privileged state targets. A history-based policy is a missing baseline: a single frame cannot determine velocity.

The Push-T planner's cost is hand-written from the known target pose, and the branched data and every plan check rely on an exact simulator; on a real robot, branches would have to come from collected play or perturbation data. The warm-start drift is explained by the plan checks on development starts and by the restart comparison, not by a controlled study of search length. With three runs per main configuration and five in the pilots, small differences between variants are within run-to-run variation, and several Push-T conclusions rest on 15-epoch models. The final comparison between planning and mini-VLA rests on three runs, in two of which mini-VLA has more successes, and only the WAM can learn from the branched data. The action-head sizes were compared on the test starts rather than on separate validation starts.

8 Conclusion

pico-WAM combines policy learning and forward prediction in a model of about a million parameters, small enough that every prediction can be checked against a simulator. In both environments, state prediction along recorded actions is far better than persistence, while accuracy under other actions depends on the training data: the point-mass model needs random-action bursts, and the Push-T model predicts the block worse than assuming it stays put until the human demonstrations are branched in the simulator.

On Push-T the world model becomes useful for control. Planning near the policy's own plan beats the reactive policies, while a stronger search, or a warm-started search that accumulates over an episode, exploits the model's errors. A larger action head was the most effective single change: with action chunks, branched data, the restarting planner and three hidden layers of 512, planning succeeds in 121 of 300 unseen episodes and mini-VLA in 116. Without state input, only the WAM learns to push the T. In the matched pilots of both environments, supervising the current state accounts for most of the auxiliary gain, more than predicting future states. Image decoding costs most of the training time and is not needed for planning. These results support evaluating factual and counterfactual accuracy, plan quality and control separately, with matched supervision controls and simple dynamics baselines.

Reproducibility. Code, stored results, and reproduction instructions are available at `https://github.com/keivalya/pico-wam`. Separate scripts run the main experiments in both environments, the supervision pilots, the affine and linear dynamics diagnostics, and the runtime benchmark. Pilot manifests record seeds and dataset and source hashes. Figures can be regenerated from stored outputs without retraining.

Assistance disclosure. AI tools assisted with code review, diagnostic experiments, literature checks, and manuscript editing.

References

[1] M. Assran, A. Bardes, D. Fan, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. arXiv:2506.09985, 2025.

[2] R. Cadene, S. Alibert, A. Soare, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. `https://github.com/huggingface/lerobot`, 2024.

[3] J. Cen, C. Yu, H. Yuan, et al. WorldVLA: Towards autoregressive action world model. arXiv:2506.21539, 2025.

[4] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion Policy: Visuomotor policy learning via action diffusion. RSS, 2023. arXiv:2303.04137.

[5] P. de Haan, D. Jayaraman, and S. Levine. Causal confusion in imitation learning. NeurIPS, 2019. arXiv:1905.11979.

[6] S. Huang, P. Kaushik, M. Chen, et al. Nano World Models: A minimalist implementation of future video prediction. arXiv:2605.23993, 2026.

[7] J. Jang, S. Ye, Z. Lin, et al. DreamGen: Unlocking generalization in robot learning through video world models. arXiv:2505.12705, 2025.

[8] M. J. Kim, Y. Gao, T.-Y. Lin, et al. Cosmos Policy: Fine-tuning video models for visuomotor control and planning. arXiv:2601.16163, 2026.

[9] L. Li, Q. Zhang, Y. Luo, et al. Causal world modeling for robot control (LingBot-VA). arXiv:2601.21998, 2026.

[10] C. Li, S. Li, Y. Yang, et al. How much planning is enough? Reducing search and computation in world-model planning (SufficientPlan). arXiv:2610.08350, 2026.

[11] Y. Liao, P. Zhou, S. Huang, et al. Genie Envisioner: A unified world foundation platform for robotic manipulation. arXiv:2508.05635, 2025.

[12] Y. Liu, H. Cui, and K. Desingh. PILOT: Guiding long-horizon world-model planning for robot manipulation with visual subgoals and policy priors. arXiv:2610.14335, 2026.

[13] Z. Lu, H. Zhai, et al. World-action models for robot learning and control: A survey. arXiv:2609.16074, 2026.

[14] K. Pandya. mini-VLA: A minimal, beginner-friendly vision-language-action model. `https://github.com/keivalya/mini-vla`, 2025.

[15] J. Qiu, Z. Chen, H. Cao, J. Shi, J. Huo, and Y. Gao. AD-WM: Action-discriminative world models for counterfactual model predictive control. arXiv:2609.30264, 2026.

[16] Q. Shen, S. Zhang, Y. Liao, et al. World action models: A survey. arXiv:2606.20781, 2026.

[17] H. Wu, Y. Jing, C. Cheang, et al. Unleashing large-scale video generative pre-training for visual robot manipulation (GR-1). arXiv:2312.13139, 2023.

[18] X. Zhang, X. Zeng, and W. Zhang. From world models to world action models: A concise tutorial for robotics. arXiv:2607.00836, 2026.

[19] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. arXiv:2411.04983, 2024.

[20] C. Zhu, R. Yu, S. Feng, et al. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. RSS, 2025. doi:10.15607/RSS.2025.XXI.015.